LIVE CODING · MÚSICA ELECTRÓNICA · JAVASCRIPT

STRUDEL

cheat sheet de ritmo, armonía y síntesis

Cada concepto musical, conectado con el método de Strudel que lo controla y con el archivo JavaScript donde vive. Todos los diagramas de patrones de este documento fueron **calculados ejecutando el motor real de Strudel**: no son dibujos aproximados.

CLAVE La fórmula: un patrón + una cadena de métodos

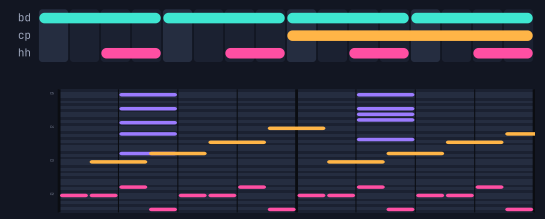
```
lead: note("c3 eb3 g3 bb3").s("sawtooth").lpf(800).lpq(6)
.attack(.01).release(.3).struct("x ~ x x").room(.4).delay(.25).gain(.8)
```

NOMBRE	QUÉ	CON QUÉ	TIMBRE	FORMA	CUÁNDO	ESPACIO	MEZCLA
`lead:` canal / pista	`note()` `n()` `chord()` alturas y armonía	`.s()` `.bank()` oscilador o sample	`.lpf()` `.lpq()` `.lpenv()` filtro = VCF	`.attack()` `.decay()`… envolvente = VCA	`.struct()` `.fast()` … ritmo y estructura	`.room()` `.delay()` `.pan()` efectos (sends)	`.gain()` `.postgain()` volumen

Leelo como un sinte modular de izquierda a derecha. Cada `.método()` recibe el patrón de la izquierda, le agrega una propiedad a cada evento y devuelve un patrón nuevo. Por eso podés agregar, sacar o reordenar eslabones sin romper nada.

START Tu primer tema: 4 capas en 7 líneas

```
setcpm(124/4)
drums: s("bd*4, [~ hh]*4, ~
cp").bank("RolandTR909")
bass:  note("<c2 c2 eb2
g1>*8").s("sawtooth").lpf(600)
chords: chord("<Cm9
Fm9>").voicing().s("triangle")
  .struct("~ x ~ ~").room(.5)
mel:   n("0 2 4 <7
6>").scale("C:minor").s("sine")
  .late(1/8).delay(.4)
```



arriba la batería (1 ciclo) · abajo el piano roll (2 ciclos): bajo, acordes, melodía

Tempo

`setcpm(124/4)` = 124 BPM en 4/4. Por defecto: `0.5 cps`, o sea 1 ciclo = 2 s ≈ 120 BPM.

Probar

Abrí **strudel.cc**, pegá un ejemplo y apretá `Ctrl+Enter`. Con `Ctrl+.` frenás todo.

Colores

ritmo · armonía · síntesis · efectos · código · hydra

Contenido

02	**Anatomía y modelo mental**	la fórmula que se repite en todo tema
03	**Strudel por dentro (JS)**	Pattern, Hap, register(), transpilador
04	**Mini-notación**	el lenguaje dentro de las comillas
05	**Ritmo I · transformaciones**	fast, off, euclid, iter, swing…
06	**Ritmo II · grooves por género**	10 patrones de batería calculados
07	**Armonía I · notas, escalas, modos**	n().scale(), teclado, intervalos
08	**Armonía II · acordes y progresiones**	chord().voicing(), arp, bajos
09	**Síntesis I · fuente y envolventes**	osciladores, ADSR, filtro, FM
10	**Síntesis II · efectos y cadena**	orden real de superdough, sidechain
11	**Azar y modulación**	señales, rand, seed, ribbon, LFOs
12	**Estilos I**	house, techno, dub techno, acid, ambient
13	**Estilos II**	DnB, trap, dembow, UK garage, lo-fi
14	**Disección: el lead del video**	línea por línea, con lo que ejecuta
15	**Hydra · visuales**	cómo se conecta con Strudel
16	**Referencia de fuentes + ruta**	función → archivo:línea, plan de práctica

Cómo usar este PDF

- Primero entendé el **modelo mental** (pág. 2–3): todo lo demás son variaciones.
- En cada página, el código de la izquierda produce el gráfico de la derecha. Cambiá un número y escuchá.
- Los grises `pattern.mjs:1905` indican dónde está la función en el código fuente de Strudel, para leerla.
- Las recetas de estilos (pág. 12–13) son plantillas: copiá, rompé, rearmá.

Recursos

`strudel.cc`	el REPL: editor + motor de audio en el navegador
`strudel.cc/learn`	docs oficiales con ejemplos que suenan
`codeberg.org/uzu/strudel`	código fuente (se mudó desde GitHub)
`npm @strudel/core`	los paquetes que se citan acá, versión 1.2.6
`hydra.ojack.xyz`	Hydra: visuales en vivo (pág. 15)

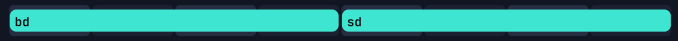
02 Anatomía y modelo mental

Cuatro ideas explican el 90% de cualquier código de Strudel. Si entendés esta página, cualquier tema se vuelve legible.

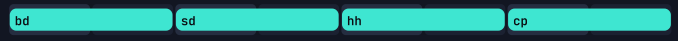
TIEMPO 1 · El ciclo es la unidad de tiempo

No hay compases ni negras: hay **ciclos**. Todo lo que escribís entre comillas se reparte **dentro de un ciclo**. Más elementos no significa más tiempo, significa elementos más cortos.

`s("bd sd")` — 2 por ciclo



`s("bd sd hh cp")` — 4 por ciclo



`s("bd sd hh cp bd sd hh cp")` — 8 por ciclo



Con `setcpm(bpm/4)`, 1 ciclo = 1 compás de 4/4. Entonces "8 por ciclo" son corcheas y "16 por ciclo" semicorcheas.

DATOS 2 · Un evento es un diccionario

Cada golpe o nota es un **Hap**: un intervalo de tiempo más un objeto de valores. Los métodos solo **agregan claves** a ese objeto. Este código:

```
note("c3 g3").s("sawtooth").lpf(800)
```

produce, en el ciclo 0, estos dos eventos (salida real de `queryArc(0,1)`):

```
0→0.5   {"note":"c3","s":"sawtooth","cutoff":800}
0.5→1   {"note":"g3","s":"sawtooth","cutoff":800}
```

El motor de audio (**superdough**) lee ese objeto y arma el sinte: `s` elige el oscilador, `cutoff` configura el filtro, etc. Fijate que `.lpf` se guarda como `cutoff`: muchos métodos son **alias** de un mismo parámetro.

PATRONES 3 · Todo argumento puede ser un patrón

Donde va un número, también puede ir un string de mini-notación o una señal. Así se automatiza cualquier parámetro sin tocar la estructura:

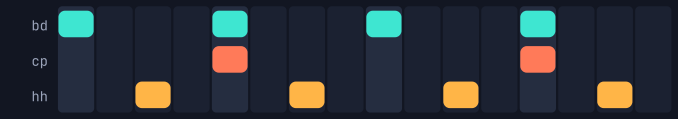
```
.lpf(800)                 // fijo
.lpf("<400 800 1600>")    // uno por ciclo
.lpf("400 2000")          // 2 por ciclo
.lpf(sine.range(300,3000).slow(8))
                          // LFO de 8 ciclos
.lpf(rand.range(200,1000)) // al azar
```

Esto pasa porque `register()` convierte todos los argumentos con `reify()` (los "patronifica") antes de aplicar la función. Lo ves en `pattern.mjs:1605`.

CAPAS 4 · Capas en paralelo

Cada línea con `nombre:` (o `$:`) es una pista que suena a la vez que las demás. Si le ponés `_` adelante o atrás (`_hats:`), la silenciás.

```
kick:  s("bd*4")
clap:  s("~ cp ~ cp")
hats:  s("[~ hh]*4")
_bass: note("c2*8").s("sawtooth") // muteado
```



La alternativa sin etiquetas es `stack()`, que da el mismo resultado:

```
stack(
  s("bd*4"),
  s("~ cp ~ cp"),
  s("[~ hh]*4")
)
```

Las etiquetas tienen una ventaja en vivo: cada capa se mutea por separado y se lee mejor.

MAPA Del sinte de hardware a Strudel

MÓDULO	STRUDEL	QUÉ HACE
VCO / oscilador	`.s("sawtooth")`	fuente de sonido; forma de onda = armónicos
Sampler	`.s("bd").bank("RolandTR909")`	reproduce grabaciones
Teclado / secuenciador	`note() n() .struct()`	alturas y cuándo suenan
VCF (filtro)	`.lpf .hpf .bpf .lpq`	esculpe el timbre
Env. de filtro	`.lpenv .lpa .lpd .lps .lpr`	mueve el cutoff en cada nota
VCA + ADSR	`.attack .decay .sustain .release`	volumen en el tiempo
LFO	`sine.range(a,b).slow(n)`	modulación lenta de cualquier parámetro
FX sends	`.room .delay .orbit`	reverb y eco por bus
Mixer	`.gain .pan .postgain`	nivel y posición
Fuente de ruido	`s("white") s("pink") s("brown")`	hats sintéticos, risers, textura
FM / operadores	`.fm(4).fmh(1.5)`	timbres metálicos y campanas
Drive / saturación	`.distort(2) .crush(6)`	calor, suciedad, lo-fi
Arpegiador	`.arp("0 1 2") · n().chord().voicing()`	recorre las notas del acorde
Sample & hold	`rand.segment(8)`	valor al azar que se sostiene por paso
Mute / solo	`_nombre: · comentar con //`	armar y desarmar el tema en vivo

MÉTODO Cómo leer código ajeno en 3 pasadas

1 · Capas. Buscá las etiquetas `nombre:` / `$:` o los `stack()`. Cada una es un instrumento. Muteá todas menos una (`_nombre:`) y escuchala sola.

2 · Esqueleto. En cada capa, encontrá el **QUÉ** (`note/n/chord/s`), el **CON QUÉ** (`.s()`) y el **CUÁNDO** (mini-notación, `struct`, `fast`). Eso ya es la canción.

3 · Adornos. Lo demás es timbre (filtro, envolvente), espacio (room, delay) y variación (`rand`, `sometimes`, `every`). Borrá adornos de a uno para entender qué aporta cada uno.

03 Strudel por dentro: el JavaScript

Lo que pasa entre que apretás Ctrl+Enter y suena. Saber esto explica por qué el código se escribe como se escribe.

PIPELINE El viaje de tu código

1 · Transpilador `@strudel/transpiler`
Reescribe tu código antes de ejecutarlo. Convierte `"…"` y `` `…` `` en `mini(…)` y `lead:` en `.p("lead")`.

2 · Mini-notación `@strudel/mini · krill.pegjs`
Una gramática PEG parsea el texto `"bd*2 [sd hh]"` a un árbol, y `patternifyAST` lo convierte en Pattern.

3 · Pattern `@strudel/core · pattern.mjs`
Un Pattern es **una función** `query(span) → Hap[]`. No guarda eventos: los **calcula** cuando alguien pregunta.

4 · Scheduler `cyclist.mjs`
Cada ~50 ms pregunta "¿qué eventos hay en esta ventana?" con `queryArc(t0, t1)`.

5 · superdough `superdough.mjs · synth.mjs`
Recibe el objeto `{s, note, cutoff…}` y construye nodos de WebAudio: oscilador, filtro, envolvente, efectos.

TRANSPILER Lo que escribís vs. lo que se ejecuta

`transpiler.mjs:71-118`

```
lead: s("saw").chord("G#m9").voicing()
  .lpf(rand.seed(3).range(200, 1000))
```

↓ salida real de `transpiler()`:

```
s(m('saw', 8))
  .chord(m('G#m9', 21))
  .voicing()
  .lpf(rand.seed(3)
  .range(200, 1000))
  .p('lead')
```

- `"comillas dobles"` y `` `backticks` `` se convierten en **mini-notación** (`m()`).
- `'comillas simples'` quedan como **string normal** de JS. Sirve para nombres, por ejemplo `.ftype('ladder')`.
- Los backticks permiten patrones multilínea, como en el video.
- `nombre:` es una *label* de JS; el transpilador la cambia por `.p('nombre')`, que registra la pista en el REPL (`repl.mjs:171`).

CLASES Las clases

`Pattern`	La clase central. `new Pattern(query)`. Todos los métodos viven en `Pattern.prototype`. `pattern.mjs:45`
`Hap`	Un evento: `whole` (duración completa), `part` (fragmento visible), `value` (el diccionario), `context`. Se llama "Hap" porque *Event* ya existe en JS. `hap.mjs:9`
`TimeSpan`	Intervalo [begin, end) en ciclos. `timespan.mjs:9`
`Fraction`	Los tiempos son **fracciones exactas** (1/3 es 1/3, no 0.333…). Así no hay deriva. `fraction.mjs`
`State`	Contexto de la consulta: controles, semillas. `state.mjs:7`
`Cyclist`	El reloj/scheduler (cps por defecto 0.5). `cyclist.mjs:10`

SEÑALES Señales: patrones continuos

```
// signal.mjs
const signal = (func) => …              // :18
const saw    = signal((t) => t % 1)     // :35
const sine = sine2.fromBipolar()        // :80
const rand = signal((t, ctl) =>         // :449
  getRandsAtTime(t, 1, …))
```

Una señal no tiene eventos: es un **valor para cada instante**. Toma la forma del patrón al que la aplicás (`.lpf(sine…)` lee la señal en el inicio de cada nota) o la discretizás con `.segment(16)`. `rand` no es `Math.random()`: es un **hash del tiempo**. El mismo instante da siempre el mismo valor, y por eso `.seed()` y `.ribbon()` pueden "congelar" el azar.



2 ciclos de cada señal, consultadas a Strudel. Rango 0..1; con `.range(a,b)` lo llevás a cualquier escala.

MÉTODOS ¿Cómo nace un método? register()

`pattern.mjs:1605`

```
// pattern.mjs:1905 — así está definido .ply()
export const ply = register('ply',
  function (factor, pat) {
    const result = pat
      .fmap((x) => pure(x)._fast(factor))
      .squeezeJoin();
    ...
  });
```

`register(nombre, fn)` hace tres cosas: (1) crea la función suelta `ply(2, pat)`, (2) agrega el método `pat.ply(2)` al prototipo, y (3) "patronifica" los argumentos para que `.ply("<1 2 4>")` funcione. El **último argumento siempre es el patrón**; por eso `.rarely(ply(2))` anda: `ply(2)` sin patrón devuelve una función que espera uno (currying).

CONTROLES Controles: registerControl()

`controls.mjs:63`

```
// controls.mjs:1204
export const { cutoff, ctf, lpf, lp } =
  registerControl(['cutoff','resonance','lpenv'],
                  'ctf', 'lpf', 'lp');
```

Los **controles** (sonido, filtro, efectos) son métodos que solo escriben una clave en el diccionario del evento. `lpf`, `lp` y `ctf` son **alias** de `cutoff`. La lista `['cutoff','resonance','lpenv']` permite escribir `.lpf("800:10:4")` para cargar cutoff, resonancia y envolvente de una sola vez.

Composers. `.set .add .sub .mul .keep .keepif` combinan dos patrones. Con `.in, .out, .mix, .squeeze` o `.reset` elegís **de cuál sale el ritmo**. `.set.out(p)` = "tomá los valores de p **y también su ritmo**" (`pattern.mjs:1020`).

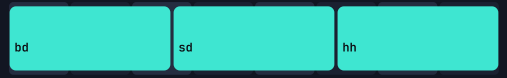
PAQUETES El ecosistema: paquetes npm

- `@strudel/core` · Pattern, tiempo, controles, señales, azar `fast stack every rand sine s note lpf`
- `@strudel/transpiler` · reescribe tu código (strings → mini, labels → .p) `transpiler evaluate`
- `superdough` · el sinte/sampler de WebAudio y los efectos `osciladores, ADSR, filtros, reverb, delay`
- `@strudel/draw` · visualizaciones en canvas `pianoroll punchcard spiral pitchwheel`
- `@strudel/mini` · parser de mini-notación (PEG) `mini m "…"`
- `@strudel/tonal` · teoría musical (usa tonal.js) `scale chord voicing transpose arp rootNotes`
- `@strudel/webaudio` · conecta los patrones con superdough · osciloscopios `scope spectrum samples`
- `@strudel/hydra` · puente con Hydra (visuales) `initHydra H`

04 Mini-notación

El lenguaje que va entre comillas dobles. Cada caja muestra el símbolo y los eventos reales que genera (cada rectángulo es un evento; el ancho es su duración).

s("bd sd hh")
secuencia: divide el ciclo en partes iguales



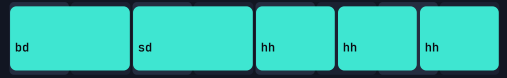
JS ≡ `seq(s("bd"), s("sd"), s("hh"))`

s("bd [sd sd] hh")
[ ] subdivide un paso



JS ≡ `seq(s("bd"), seq(s("sd"), s("sd")), s("hh"))`

s("bd sd . hh hh hh")
. agrupa sin corchetes



JS ≡ `s("[bd sd] [hh hh hh]")`

s("bd*2 sd")
*n repite más rápido



JS ≡ `seq(s("bd").fast(2), s("sd"))`

s("bd sd/2")
/n más lento: aparece cada n ciclos



JS ≡ `seq(s("bd"), s("sd").slow(2))`

s("<bd sd cp>")
< > alterna uno por ciclo



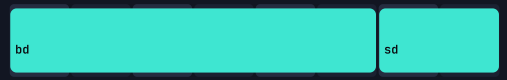
JS ≡ `cat(s("bd"), s("sd"), s("cp"))`

s("bd ~ sd -")
~ o - silencio



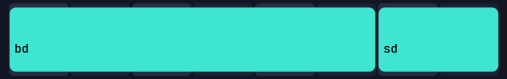
JS ≡ `seq(s("bd"), silence, s("sd"), silence)`

s("bd _ _ sd")
_ estira el evento anterior



JS ≡ `s("bd@3 sd")`

s("bd@3 sd")
@n peso (duración relativa)



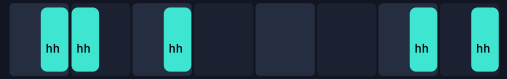
JS ≡ `stepcat([3, s("bd")], [1, s("sd")])`

s("bd!3 sd")
!n replica el paso n veces



JS ≡ `seq(s("bd"), s("bd"), s("bd"), s("sd"))`

s("hh*16?")
? 50% de probabilidad (?0.2 = 20%)



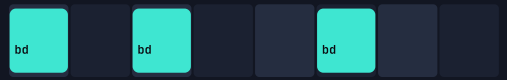
JS ≡ `s("hh*16").degradeBy(0.5)`

s("bd(3,8)")
(k,n) ritmo euclídeo: k golpes en n pasos



JS ≡ `s("bd").euclid(3, 8)`

s("bd(3,8,2)")
(k,n,r) euclídeo rotado r pasos



JS ≡ `s("bd").euclidRot(3, 8, 2)`

s("bd*4, hh*8")
, apila capas (polifonía)



JS ≡ `stack(s("bd*4"), s("hh*8"))`

s("{bd sd, hh hh hh}")
{ } polimetría: alinea el paso, no el ciclo



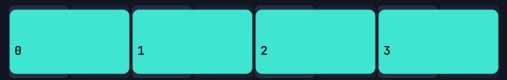
JS ≡ `stack(s("bd sd").pace(2), s("hh hh hh").pace(2))`

s("{bd sd hh}%4")
{ }%n n pasos por ciclo, recorre la lista



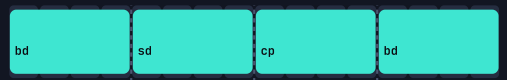
JS ≡ `s("bd sd hh").pace(4)`

n("0 .. 3")
.. rango de números



JS ≡ `n(run(4))`

s("bd | sd | cp")
| elige una opción al azar por ciclo



JS ≡ ≈ `randcat(…)` (azar por ciclo)

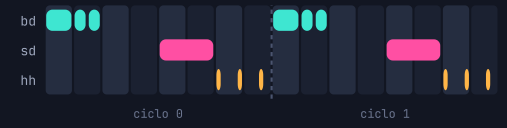
Valores con ":"

verificado: cada equivalencia JS ≡ produce exactamente los mismos eventos.

`"bd:3"` elige el sample nº 3 del banco *bd*. `"c3:0.5" + .as("note:gain")` carga dos parámetros a la vez.

Anidar

Todo se combina: `"<bd [bd bd]>*2 [~ sd]@2 hh(3,8)"`. Un `< >` adentro de `[ ]` alterna en cada pasada.

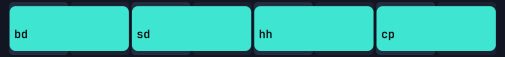


05 Ritmo I · transformaciones temporales

Métodos que reescriben el CUÁNDO. Todos toman un patrón y devuelven otro, así que se encadenan. Gráficos calculados con Strudel.

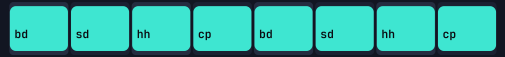
`s("bd sd hh cp")` — patrón base

el original: 4 pasos por ciclo



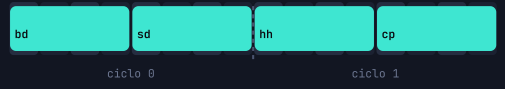
`s("bd sd hh cp").fast(2)` — pattern.mjs:1924

comprime ×2: entra dos veces por ciclo



`s("bd sd hh cp").slow(2)` — pattern.mjs:1958

estira ×2: ocupa dos ciclos



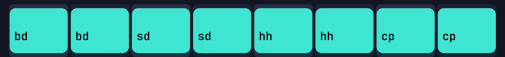
`s("bd sd hh cp").rev()` — pattern.mjs:2259

invierte el orden dentro del ciclo



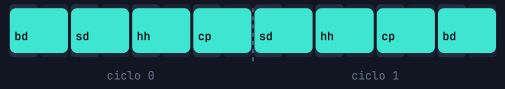
`s("bd sd hh cp").ply(2)` — pattern.mjs:1905

cada evento se repite n veces en su lugar



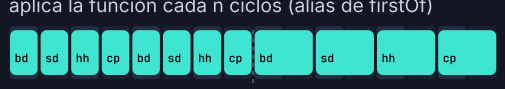
`s("bd sd hh cp").iter(4)` — pattern.mjs:2499

rota el comienzo en cada ciclo



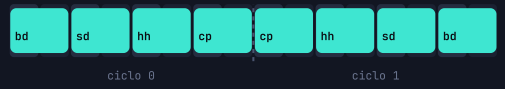
`s("bd sd hh cp").every(2, x=>x.fast(2))` — pattern.mjs:2022

aplica la función cada n ciclos (alias de firstOf)



`s("bd sd hh cp").palindrome()` — pattern.mjs:2340

ida y vuelta: alterna normal y rev



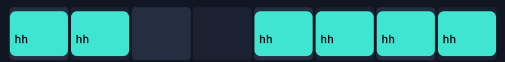
`s("bd*4").struct("x ~ x x")` — composers

impone un ritmo: x = suena, ~ = silencio



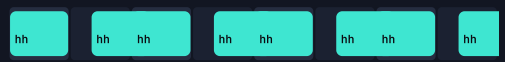
`s("hh*8").mask("1 0 1 1")` — composers

apaga zonas sin cambiar la grilla



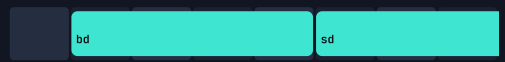
`s("hh*8").swingBy(1/3, 4)` — pattern.mjs:2184

swing: atrasa la 2ª mitad de cada 1/4



`s("bd sd").late(0.125)` — pattern.mjs:2081

corre todo hacia atrás (early = adelanta)



`s("bd").euclid(5, 8)` — euclid.mjs:138

k golpes repartidos en n pasos (Bjorklund)



`s("bd").beat("0,3,6,10,12", 16)` — pattern.mjs:3521

golpes en posiciones exactas de una grilla



`s("bd sd hh cp").linger(0.25)` — pattern.mjs:2151

repite el primer cuarto: efecto "stutter"



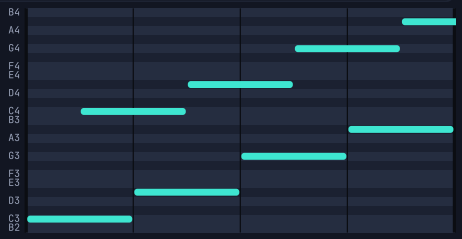
`n("0 1 2 3").chunk(4, x=>x.add(n(7)))` — pattern.mjs:2571

transforma 1 de n trozos, uno distinto por ciclo



CAPAS Desfase y capas

```
n("0 2 4 6")
  .off(1/8, x=>x.add(n(7)))
  .scale("C:minor")
```



Copia el patrón corrido 1/8 de ciclo y 7 grados más arriba (= una octava en una escala de 7 notas).

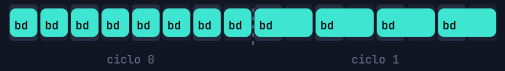
`off` = `superimpose` + `late` (pattern.mjs:2236). `jux(rev)` pone el original a la izquierda y la versión transformada a la derecha (:2379).

PATRONES Parámetros que cambian por ciclo

```
s("bd sd hh cp")
  .fast("<1 2 4>")  // ×1, ×2, ×4
```




```
s("bd*4").every(4, x=>x.ply(2))
```



TIPS Reglas de oro del ritmo

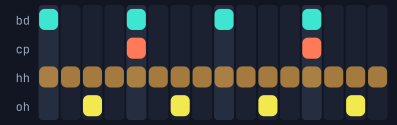
- **fast/slow** cambian la densidad; **ply** cambia la densidad *sin* mover los golpes.
- **struct** y **mask** separan el *qué* del *cuándo*: definís notas por un lado y ritmo por el otro (eso hace el `.set.out` del video).
- **every / iter / chunk / palindrome** generan variación por ciclo: evitan que un loop canse.
- Los argumentos también son patrones: `.fast("<1 2 [4 8]>")`.
- Ojo: `.add(7)` sobre `n(...)` no funciona (es un "control pattern"). Usá `.add(n(7))` o sumá adentro: `n("0 2".add(7))`.

FIGURA	FRACCIÓN DE CICLO	MINI-NOTACIÓN (1 CICLO = 1 COMPÁS)	MÉTODO
Redonda	1	`"bd"`	`.slow(1)`
Negras	1/4	`"bd*4"`	`.fast(4)`
Corcheas	1/8	`"hh*8"` · contratiempo: `"[~ hh]*4"`	`.late(1/8)`
Semicorcheas	1/16	`"hh*16"`	`.ply(2)` sobre corcheas
Tresillos de negra	1/6	`"hh*6"` · `"[hh hh hh]*2"`	`.fast(6)`
Puntillo (3+3+2)	3/8 3/8 2/8	`"bd@3 bd@3 bd@2"` = `"bd(3,8)"`	`.euclid(3,8)`
Compás de 3/4	1/3	`"bd hh hh"` + `setcpm(bpm/3)`	—

06 Ritmo II · grooves por género

Patrones de batería en una grilla de 16 pasos (1 ciclo = 1 compás). Las columnas claras marcan los tiempos 1-2-3-4. La opacidad indica el gain.

House / Disco — 124 BPM



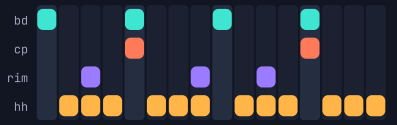

```
$: s("bd*4")
$: s("~ cp ~ cp")
$: s("[~ oh]*4")
$: s("hh*16").gain(.4)
```

four-on-the-floor, palmas en 2 y 4, hat abierto a contratiempo. Probá .bank("RolandTR909").

Sumale un bajo o acordes:

```
note("[~ c2]*4")
  .s("sawtooth").lpf(500)
```

Techno — 132 BPM



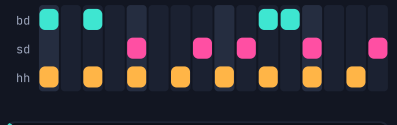

```
$: s("bd*4")
$: s("cp").beat("4,12", 16)
$: s("[~ hh hh hh]*4")
$: s("rim").beat("2,7,10", 16)
```

bombo continuo, hats en semicorcheas con hueco en el tiempo, rim sincopado.

Sumale un bajo o acordes:

```
note("c1*16").s("sawtooth")
  .lpf(sine.range(200,800)
    .slow(8))
  .decay(.1).sustain(0)
```

Breakbeat (Amen-ish) — 136 BPM



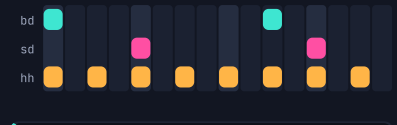

```
$: s("bd").beat("0,2,10,11", 16)
$: s("sd").beat("4,7,9,12,15", 16)
$: s("hh*8")
```

bombo sincopado y caja desplazada: el corazón del jungle y el big beat.

Sumale un bajo o acordes:

```
note("<c2 [~ c2] bb1 g1>")
  .s("square").lpf(800)
```

Drum & Bass — 174 BPM



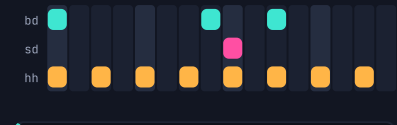

```
$: s("bd").beat("0,10", 16)
$: s("sd").beat("4,12", 16)
$: s("hh*8")
```

two-step: bombo en el 1 y en el "y" del 3, caja en 2 y 4.

Sumale un bajo o acordes:

```
note("<c1 eb1>/2")
  .s("supersaw")
  .lpf(300).lpq(5) // reese
```

Trap / Hip-hop — 140 BPM



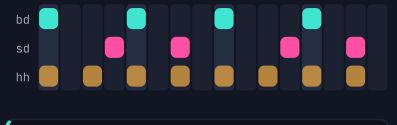

```
$: s("bd").beat("0,7,10", 16)
$: s("sd").beat("8", 16)
$: s("hh*8")
  .ply("<1 [1 1 1 2] 1 [1 3]>")
```

half-time: la caja cae en el 3. Los hats hacen redobles con ply.

Sumale un bajo o acordes:

```
note("c1").beat("0,7,10",16)
  .s("sine") // 808
  .decay(.8).sustain(0)
```

Dembow / Reggaetón — 95 BPM



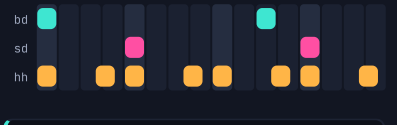

```
$: s("bd*4")
$: s("[~ ~ ~ sd ~ ~ sd ~]*2")
$: s("hh*8").gain(.5)
```

bombo en negras + caja en tresillo 3-3-2 (x··x··x·).

Sumale un bajo o acordes:

```
note("<a1 f1 c2 g1>")
  .s("sawtooth").lpf(600)
  .struct("x ~ ~ x ~ ~ x ~")
```

UK Garage / 2-step — 132 BPM



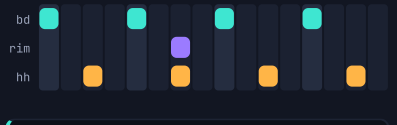

```
$: s("bd").beat("0,10", 16)
$: s("~ sd ~ sd")
$: s("hh*8").swingBy(1/3, 4)
```

bombo que saltea el 3 y hats con swing marcado.

Sumale un bajo o acordes:

```
chord("<Fm9 Bbm9>")
  .voicing().s("sawtooth")
  .struct("~ x ~ ~ x ~ ~ ~")
  .lpf(1500)
```

Dub Techno — 118 BPM



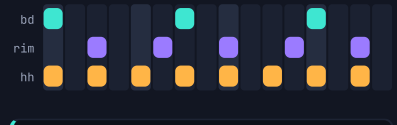

```
$: s("bd*4")
$: s("[~ hh]*4")
$: s("rim").beat("6", 16)
```

bombo suave y hats a contratiempo; el groove lo dan los acordes con delay (pág. 12).

Sumale un bajo o acordes:

```
chord("Cm7").voicing()
  .s("sawtooth").lpf(700)
  .struct("~ x ~ ~")
  .delay(.6).room(.7)
```

Afro / Tresillo — 120 BPM



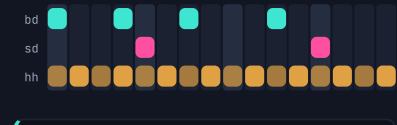

```
$: s("bd(3,8)")
$: s("rim(5,16,2)")
$: s("hh*8")
```

el tresillo (3,8) y la clave de son (5,16): ritmos euclídeos.

Sumale un bajo o acordes:

```
n("0 2 4 [2 4]")
  .scale("D:dorian")
  .s("triangle").fast(2)
```

Electro — 128 BPM




```
$: s("bd").beat("0,3,6,10", 16)
$: s("~ sd ~ sd")
$: s("hh*16").gain("[.4 .8]*8")
```

herencia de la 808: bombo sincopado, caja en 2 y 4, hats acentuados.

Sumale un bajo o acordes:

```
note("c2 c2 [c3 c2] eb2")
  .s("square").lpf(1200)
  .decay(.2).sustain(0)
```

TIPS Cómo sonar a máquina real

- `.bank("RolandTR909")` house y techno · `"RolandTR808"` trap, electro y dembow · `"LinnDrum"` pop 80s. Por ejemplo: `s("bd*4").bank("RolandTR909")`.
- Las abreviaturas son estándar: `bd` bombo, `sd` caja, `hh` / `oh` hat cerrado/abierto, `cp` palmas, `rim` aro, `lt mt ht` toms, `cr rd` platos.
- **Velocity humana:** `.gain("[.8 .5 .7 .5]*4")` o `.velocity(rand.range(.6,1))`.
- **Groove:** `.swingBy(1/6, 4)` sutil, `1/3` marcado. `.nudge(0.01)` retrasa unos milisegundos.
- **Variación:** `.sometimes(x=>x.ply(2))`, `.every(4, x=>x.fast(2))` para el fill del 4º compás.
- **Sidechain:** `.duckorbit(2)` en el bombo hace "respirar" a lo que suena en el orbit 2 (pág. 10).
- **BPM:** `setcpm(132/4)` al principio del código.
- La grilla de 16 pasos es solo una convención: `"bd(5,12)"` también vale.

07 Armonía I · notas, intervalos, escalas

De la teoría al código: note() da alturas absolutas; n() + .scale() da GRADOS dentro de una escala, y así nunca desafinás.

ALTURA note() vs n() vs freq()

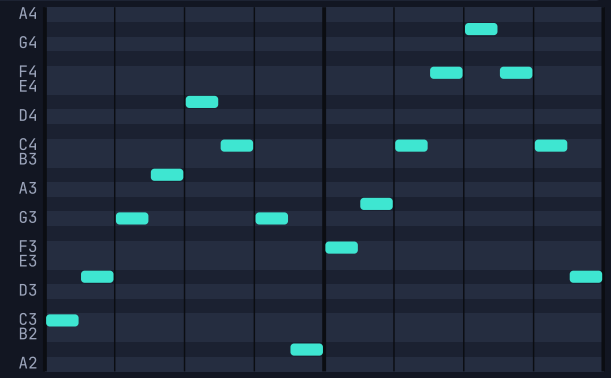
`note("c3 eb3 g3")`	nombre + octava. `#` y `b` para alteraciones: `"f#3"` = `"gb3"`.
`note("48 51 55")`	lo mismo en números MIDI. C4 = 60; cada número es un semitono.
`n("0 2 4")`	**índice**: con `.scale()` es un grado de la escala; con samples elige el número de sample (`s("bd").n(3)`).
`freq("220 330")`	frecuencia en Hz, para microtonalidad.



Arriba: `note("c4 e4 g4")` = MIDI 60, 64, 67. El punto marca la tónica. `controls.mjs:366 (n) · :388 (note)`

GRADOS Grados: cómo piensa n().scale()

```
n("0 2 4 <6 7> 9 7 4 -1")
  .scale("<C:minor F:dorian>")
  .s("triangle")
```

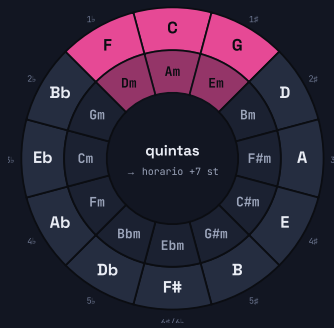


- El grado **0** es la tónica (en octava 3). **7** es la tónica una octava arriba y los **negativos** bajan.
- La escala también es un patrón: cambiala por ciclo y la melodía "se adapta" sola.
- `.scaleTranspose(1)` sube un *grado* · `.transpose(1)` sube un *semitono* (`tonal.mjs:167 · :114`).

SEMITONOS Intervalos: la distancia lo es todo

ST	NOMBRE		CARÁCTER
0	unísono	P1	misma nota
1	2ª menor	m2	tensión, terror
2	2ª mayor	M2	paso, suspenso
3	3ª menor	m3	triste → acorde menor
4	3ª mayor	M3	alegre → acorde mayor
5	4ª justa	P4	abierta, "himno"
6	tritono	TT	inestable, disonante
7	5ª justa	P5	poder, "power chord"
8	6ª menor	m6	melancólica
9	6ª mayor	M6	dulce
10	7ª menor	m7	blues, funk, groove
11	7ª mayor	M7	soñadora, jazz/lo-fi
12	octava	P8	la misma nota, más aguda

Sumás semitonos con `.transpose(7)` o `.add(note(7))`. Una 3ª menor es `note("c3 eb3")`.



Círculo de quintas

Afuera las tonalidades mayores y adentro sus **relativas menores** (mismas notas). Los vecinos comparten 6 de 7 notas, así que **suenan bien juntos**: en C, los acordes de F, C y G (y Dm, Am, Em) son la "familia". Para modular suave, movete un paso; para un cambio dramático, cruzá el círculo.

Ej.: `chord("<C G Am F>")` usa solo la zona marcada.

MODOS Escalas y modos: n("0 .. 7").scale("C:…")

"C:major" — mayor / jónico



luminoso, pop, house "feliz"

"C:minor" — menor natural / eólico



oscuro, techno, trance, DnB

"C:dorian" — dórico



menor con 6ª mayor: funk, deep house, jazz

"C:phrygian" — frigio



2ª menor: tenso, flamenco, dark techno

"C:lydian" — lidio



4ª aumentada: flotante, ambient, cine

"C:mixolydian" — mixolidio



mayor con 7ª menor: rock, blues, disco

"C:minor:pentatonic" — pentatónica menor



5 notas: no hay notas "malas"

"C:harmonic:minor" — menor armónica



7ª mayor: dramática, oriental

Notas reales que devuelve Strudel para los grados 0–6. Más escalas: `blues`, `major:pentatonic`, `melodic:minor`, `whole:tone`, `locrian`, `chromatic` … (son los nombres de tonal.js) · `tonal.mjs:268`

ESTILO	ESCALAS TÍPICAS	POR QUÉ
Techno / Acid	`minor` · `phrygian`	oscuro; la 2ª menor del frigio da tensión
Deep house	`dorian` · `minor`	menor pero cálido (6ª mayor)
Trance / EDM	`minor` · `harmonic:minor`	épico; la 7ª mayor empuja a resolver
Ambient	`lydian` · `major:pentatonic`	sin tensiones fuertes, flota
Hip-hop / Lo-fi	`minor:pentatonic` · `dorian`	melodías que nunca chocan
Reggaetón	`minor` · `harmonic:minor`	progresión andaluza (pág. 8)

08 Armonía II · acordes, voicings, progresiones

chord("...").voicing() convierte un símbolo de cifrado en notas bien distribuidas. Notas y piano rolls calculados con @strudel/tonal.

ACORDES Diccionario de acordes (cifrado americano, raíz C)

C mayor · 1-3-5

E3 C4 E4 G4 C5 · estable, pop

Cm menor · 1-♭3-5

G3 C4 Eb4 G4 C5 · oscuro

C7 dominante · +♭7

E3 Bb3 E4 G4 C5 · tensión → resuelve

C^7 maj7 · +7

C3 E4 G4 B4 · soñador, lo-fi

Cm7 m7 · +♭7

Bb3 Eb4 G4 Bb4 C5 · deep house

Cm9 m9 · +♭7+9

Eb3 Bb3 D4 G4 C5 · house, garage

Csus sus4 · 1-4-5

C4 F4 G4 C5 · abierto, trance

Co disminuido · 1-♭3-♭5

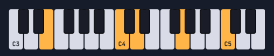
Eb3 C4 Eb4 Gb4 C5 · inquietante

Ch7 semidism. ø · 1-♭3-♭5-♭7

Bb3 Eb4 Gb4 Bb4 C5 · jazz, cine

C6 sexta · +6

C3 G3 C4 E4 A4 · retro, bossa

Cadd9 add9 · +9

E3 C4 D4 G4 C5 · brillante, indie

C7b9 7♭9 · dom. alterado

Bb3 Db4 E4 Bb4 C5 · tensión máxima

Notas reales que devuelve `chord("X").voicing()` (diccionario `ireal` por defecto; anclado debajo de C5). Alias aceptados: `m` = `-`, `^7` = maj7, `h7` = ø, `o` = dim. `voicings.mjs:196 · ireal.mjs`

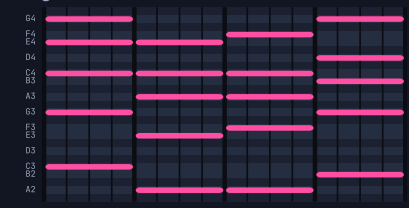
VOICING Controlar el voicing

```
chord("<C Am F G>")
  .anchor("c5")    // nota tope de referencia
  .mode("below")   // below · above · duck · root
  .dict("ireal")   // diccionario: ireal · lefthand
  .voicing()
```

anchor c5 (default)



anchor("g4")



El voicing busca **la inversión más cercana** al ancla, así los acordes se mueven poco (*voice leading*). Con `n("0 1 2 3").chord(…).voicing()` recorrés las notas del acorde como una escala: es un arpegiador.

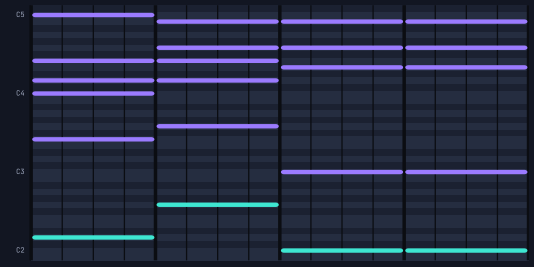
PROGRESIONES Progresiones clásicas · acordes + bajo (rootNotes)

i – VI – III – VII `"<Am F C G>"`



EDM, trance, pop: la más usada

ii – V – I `"<Dm7 G7 C^7 C^7>"`



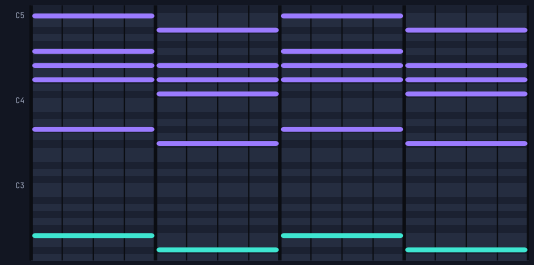
jazz, lo-fi, neo-soul

i – ♭VII – ♭VI – V `"<Am G F E>"`



andaluza: reggaetón, flamenco

i9 – ♭VII9 `"<Fm9 Ebm9>"`



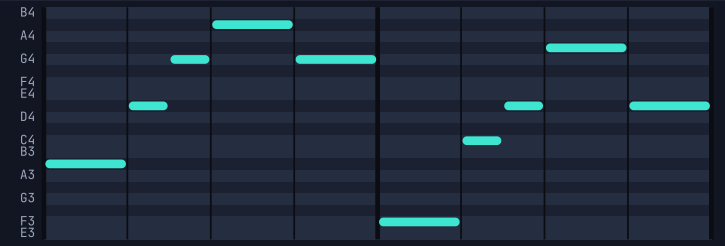
deep house, garage (2 acordes)

```
$: chord("<Am F C G>").voicing().s("sawtooth").lpf(1500)
$: chord("<Am F C G>").rootNotes(2).s("square").lpf(400)  // bajo = raíz en octava 2
```

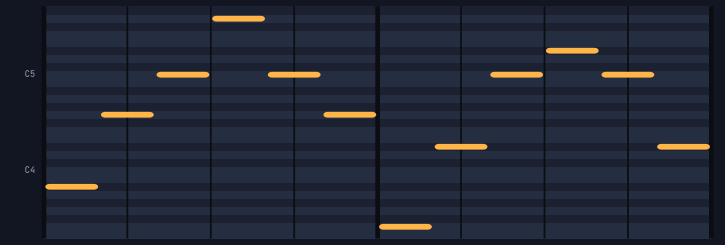
ARP Arpegios y melodías sobre acordes

pattern.mjs:966

```
chord("<Cm7 Fm7>").voicing()
  .arp("0 [1 2] 3 2")     // índices de nota del acorde
```



```
n("0 2 4 7 4 2").chord("<Cm7 Fm7>").voicing()  // voicing como escala
```



- **Números romanos:** I = acorde sobre el grado 1. Mayúscula = mayor, minúscula = menor.
- **Tensión → reposo:** V7 → I. En menor, usá E7 (no Em) para volver a Am con fuerza.
- **Truco house:** un solo acorde m9 con `.struct("~ x ~ ~")` y filtro que se abre = 8 compases resueltos.

09 Síntesis I · fuente, envolventes, filtro

Así se ve tu sintetizador por dentro. Lo que en hardware son perillas, acá son métodos, y cada uno acepta patrones y señales.

VCO Osciladores: `.s("…")`

"sine" seno



solo la fundamental: subgraves, 808, pads puros

"triangle" triangular



armónicos impares débiles: suave, flauta

"square" cuadrada



impares fuertes: hueco, chiptune, bajos 8-bit. Con .pw() es pulse

"sawtooth" diente de sierra



todos los armónicos: brillante. La favorita para filtrar

"supersaw" supersaw



varios saws desafinados (.unison .detune .spread): trance, EDM

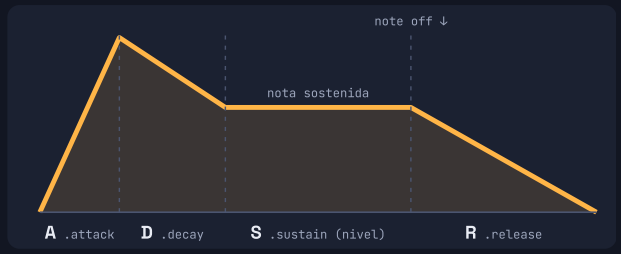
"white" ruido



white · pink · brown · crackle: hats, risers, viento

Forma de onda (arriba) y armónicos 1–16 (abajo). Alias: `saw`, `tri`, `sqr` (`synth.mjs:23-27`). Un timbre brillante tiene muchos armónicos, y el filtro está para sacárselos.

VCA ADSR: la envolvente de volumen (VCA)



pluck pad órgano bajo



```
note("c3 eb3 g3 bb3").s("sawtooth")
  .attack(0).decay(.15).sustain(0) // pluck
  .release("<.1 1>")               // por ciclo
```

PARÁMETRO	UNIDAD	DEFAULT SYNTH	USOS
`.attack(a) .att`	segundos	0.001	0 = golpe · 1+ = pad que entra lento
`.decay(d) .dec`	segundos	0.05	cuánto tarda en caer al sustain
`.sustain(s) .sus`	**nivel 0–1**	0.6	0 = percusivo (pluck)
`.release(r) .rel`	segundos	0.01	cola después del note-off

Atajo: `.adsr(".01:.2:.5:.3")` (`controls.mjs:2572`). Defaults reales: `synth.mjs:50`. La duración de la nota (cuándo llega el note-off) la manejan `.clip()` / `.legato()`.

Diferencia con tu sinte de hardware: en Strudel **cada nota** crea su propia voz, con oscilador, filtro y envolventes propios. La polifonía es prácticamente ilimitada (hasta 128 voces por defecto, `superdough.mjs:33`), y cada nota puede tener *otros* valores: `.attack("0 .5")` alterna golpe y fade entre notas.

MÁS FM, pitch y más fuentes

`controls.mjs:473-1914`

`.fm(2) .fmh(1.5)`	FM: un oscilador modula la frecuencia de otro. `fm` = índice (brillo, metal), `fmh` = relación de armónicos (enteros = armónico; decimales = campana)
`.fmenv .fmattack`	envolvente propia de la FM: ataques metálicos que se apagan
`.penv(12).pdecay(.1)`	envolvente de **pitch** en semitonos: el "pew" del láser o el golpe del bombo 808
`.vib(4).vibmod(.5)`	vibrato: frecuencia (Hz) y profundidad (semitonos)
`.unison .detune .spread`	voces, desafinación y apertura estéreo del supersaw
`.coarse .crush`	lo-fi: baja la frecuencia de muestreo / los bits

VCF Filtros: esculpir armónicos (VCF)

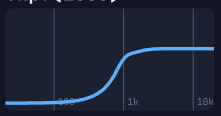
`controls.mjs:1204`

`.lpf(1000)`



pasa-bajos: oscurece (punteado: 300 Hz)

`.hpf(1000)`



pasa-altos: adelgaza

`.bpf(1000)`



pasa-banda: "teléfono"

`.lpf(1000).lpq(10)`



`.lpf(sine.range(300,3000).slow(4))`



barrido automático (LFO)

`.lpq(10)`	resonancia: realza la frecuencia de corte (el "chillido" acid)
`.ftype('ladder')`	tipo de filtro: `12db`, `ladder` (más agresivo), `24db`
`.lpf("800:10")`	cutoff y resonancia juntos, separados por `:`
`.vowel("a e i")`	filtro de formantes: suena como una voz

FENV Envolvente de filtro: el "wow"




```
note("c2*8").s("sawtooth")
  .lpf(300).lpenv(4)    // sube 4 octavas
  .lpa(.05).lpd(.25)    // A y D del filtro
  .lps(0)               // S del filtro
```

`lpenv` se mide en **octavas** (`2 ** env` en `helpers.mjs:261`). Defaults del filtro: A .005 · D .14 · S 0 · R .1. También: `.hpenv`, `.bpenv`.

10 Síntesis II · efectos, cadena de señal, mezcla

El orden de los efectos no lo elegís vos: superdough lo fija. Este es el orden real, leído del código fuente.

CADENA La cadena real de superdough (orden fijo)

Fuente	Filtros	Color	Modulación	Dinámica	Estéreo	Nivel	Sends (orbit)
s · note · bank attack…release	lpf → hpf → bpf	vowel · coarse crush · shape · distort	tremolo	compressor	pan · phaser	postgain	delay · room → djf → salida

Leído de `superdough.mjs:682-963`. Cada evento crea su propia cadena; **delay y reverb son buses compartidos** por `orbit`: `.room()` y `.delay()` son cuánto mandás (send) al bus, no efectos por nota. Por eso en el código da igual dónde escribas `.room()`: el orden es siempre este.

ESPACIO Espacio: reverb y delay

`controls.mjs:1733 · 2196`

`.room(.4)`	cuánto mandás a la reverb (0–1)
`.roomsize(4)` `.size`	tamaño 0–10 (también `.roomfade`, `.roomlp`, `.roomdim`)
`.delay(.5)`	cuánto mandás al delay
`.delaytime(.375)` `.dt`	tiempo en segundos. Si no lo ponés, usa **3/16 de ciclo** (corchea con puntillo, sincronizado al tempo) · `superdough.mjs:194`
`.delayfeedback(.6)` `.dfb`	repeticiones (< 1, o se autooscila)
`.delay(".5:.375:.6")`	los tres juntos
`.orbit(2)`	bus separado: otra reverb y otro delay

COLOR Color: saturación y lo-fi

`controls.mjs:858-2323`

`.distort(2)`	waveshaper (útil de 0 a 10); `"2:.5"` = cantidad:volumen de salida
`.shape(.5)`	versión vieja (deprecada): preferí `distort`
`.crush(4)`	bitcrusher: 1 = destruido, 16 = limpio
`.coarse(8)`	divide el sample rate (1 = original): aliasing, chiptune
`.vowel("a")`	formantes: a e i o u
`.phaser(2)`	phaser (velocidad); `.phaserdepth(0-1)`
`.tremolo(4)`	modulación de volumen (velocidad en Hz)

MIX Mezcla y dinámica

`controls.mjs:411-2364`

`.gain(.8)`	volumen de entrada (antes de la cadena)
`.velocity(.7)`	intensidad 0–1; se multiplica con gain (`superdough.mjs:611`)
`.postgain(1.2)`	volumen final, después de los efectos
`.pan(.3)`	0 = izquierda · .5 = centro · 1 = derecha
`.compressor(…)`	`"-20:20:10:.002:.02"` = umbral:ratio:knee:ataque:release
`.jux(rev)`	L = original, R = transformado: estéreo instantáneo

Ojo con la suma. Diez capas a gain 1 saturan. Empezá con el bombo en `.gain(1)` y el resto entre .3 y .7.

SIDECHAIN Sidechain / ducking: el "bombeo" del house

```
pad:  chord("<Cm9 Fm9>").voicing()
        .s("supersaw").lpf(1200)
        .orbit(2)            // bus 2
kick: s("bd*4").duckorbit(2) // cada bombo lo baja
        .duckattack(.2).duckdepth(1)
```



`duckorbit` baja el volumen de otro orbit en cada golpe; `duckattack` es el tiempo que tarda en volver y `duckdepth` cuánto baja (`controls.mjs:971-1038`).

PRESETS Recetas de timbre

SONIDO	CADENA	IDEA
pluck	`.s("sawtooth").decay(.15).sustain(0).lpf(2000)`	sin sustain = percusivo
pad	`.s("supersaw").attack(1).release(2).lpf(1500).room(.8)`	entra y sale lento, mucho espacio
acid	`.s("sawtooth").lpf(400).lpq(12).lpenv(3).ftype('ladder')`	resonancia + envolvente de filtro
sub	`note("c1 c2").s("sine").lpf(200)`	octavas 1–2, sin armónicos
reese	`.s("supersaw").detune(.5).lpf(300).lpq(5)`	saws desafinados: DnB
stab	`.s("square").decay(.1).sustain(0).delay(.5).room(.6)`	corto + eco = dub
bell	`.s("sine").fm(4).fmh(3.5).decay(1).sustain(0)`	FM con relación no entera = metal
808	`.s("sine").penv(12).pdecay(.05).decay(.8).sustain(0)`	golpe de pitch + cola larga
lo-fi	`.crush(6).coarse(4).lpf(3000)`	menos bits, menos sample rate

Mapa de frecuencias: cada instrumento tiene "su" zona. Limpiá lo que no le corresponde con `.hpf()` o `.lpf()` y la mezcla se aclara sola.

sub	graves	medios	presencia	aire
20-60 Hz	60-250 Hz	250-2k Hz	2k-6k Hz	6k-20k Hz
bombo · 808 · sub	bajo · cuerpo del bombo	acordes · voces · leads	ataque · caja · claridad	hats · brillo · reverb

Ej.: `.hpf(150)` en pads y acordes deja lugar al bajo; `.lpf(8000)` en la reverb evita que chille.

11 Azar y modulación

LFOs, aleatoriedad y probabilidad: lo que hace que un loop de 1 compás suene vivo durante 5 minutos. Señales consultadas a Strudel (2 ciclos).

LFO Señales continuas (rango 0..1)

sine signal.mjs



cosine signal.mjs



saw signal.mjs



isaw signal.mjs



tri signal.mjs



square signal.mjs



perlin signal.mjs



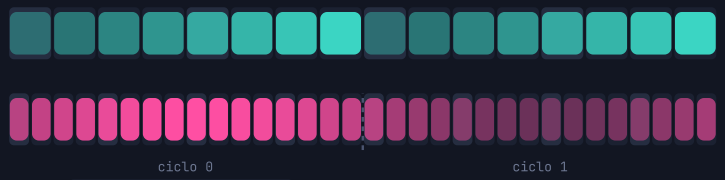
rand signal.mjs



```
.lpf(sine.range(300, 3000).slow(8))    // LFO de 8 ciclos
.pan(perlin.range(.2, .8))              // deriva orgánica
.gain(saw.range(.2, 1).fast(4))         // rampa por negra
```

`.range(a,b)` escala · `.slow(n)` baja la frecuencia del LFO · `.segment(n)` lo convierte en n escalones · `.rangex` escala exponencial (ideal para Hz).

La señal toma la forma del patrón: se lee en el inicio de cada evento. Acá, `s("hh*16").gain(saw.fast(2))`. La opacidad es el gain que recibió cada hat:



abajo: `.gain(sine.slow(2))` durante 2 ciclos: un "swell" de volumen.

RANDOM Azar discreto

signal.mjs:488-624

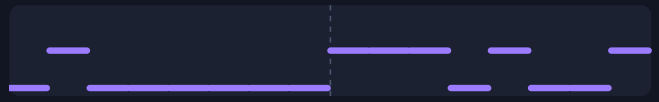
`s("hh*8").n(irand(8))` entero al azar 0–7



`s("hh*8").n(choose(0, 3, 7))` uno de la lista



`s("hh*8").n(wchoose([0,3],[4,2],[7,1]))` con pesos 3 : 2 : 1



`n("0 1 2 3 4 5 6 7").shuffle(8)` permuta sin repetir



También: `randcat(a,b,c)` elige un patrón por ciclo · `.scramble(n)` sortea con repetición.

RIBBON Congelar el azar: seed y ribbon

```
n(irand(8).segment(4)).scale("C:minor")
  .ribbon(1337, 2)  // loop de 2 ciclos desde el 1337
```



- `rand` depende del tiempo: el ciclo 1337 siempre suena igual. `ribbon(offset, ciclos)` = `early(offset)` + reinicio en loop (`pattern.mjs:2678`).
- `.seed(n)` cambia la "semilla": otra secuencia, igual de reproducible.
- **Flujo de trabajo:** generá con azar → probá offsets → quedate con el que te gusta. Es lo que hace la autora del video con `rib(22,1)` y `rib(29,1)`.

PROB Probabilidad: aplicar una función a veces




```
s("hh*16").sometimes(x => x.speed(2))
s("bd*4").rarely(ply(2))              // del video
s("hh*8").sometimesBy(.3, x => x.gain(.3))
s("hh*16").degradeBy(.4)              // quita ~40%
n("0 2 4").someCycles(x => x.rev())   // decide por ciclo
```

Valores leídos de `signal.mjs:783-923`. Abajo, `s("hh*16").degradeBy(.4)` en 2 ciclos: el azar se decide **por evento**.



ESTRUCTURA Forma del tema: arrange y mask

```
arrange(
  [4, s("bd*4")],                    // intro
  [8, stack(s("bd*4"), s("hh*8"))], // A
  [4, s("~ cp")]                     // break
)
```



16 ciclos de un vistazo (cada columna es un ciclo).

```
// los hats entran en el ciclo 5:
hats: s("hh*8").mask("<0!4 1!12>")
// el bajo entra en el ciclo 9:
bass: note("c2*8").mask("<0!8 1!8>")
```

En vivo lo más común es comentar y descomentar capas (`_bass:`). `arrange()` y `mask()` sirven para dejar la estructura **escrita** (`pattern.mjs:1469`).

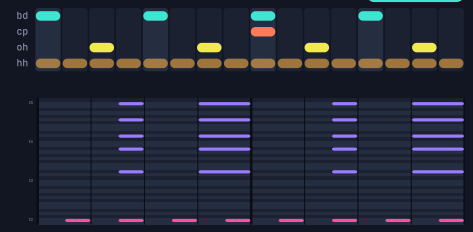
12 Estilos I · house, techno, dub, acid, ambient

Plantillas completas para copiar en strudel.cc. Cada una fue evaluada con el motor real: los gráficos muestran lo que suena. Usan samples y sintes por defecto.

House

124 BPM

```
setcpm(124/4)
drums: s("bd*4, [~ oh]*4, ~ cp")
  .bank("RolandTR909")
hats: s("hh*16").bank("RolandTR909")
  .gain(.35)
bass: note("<c2 c2 f1 g1>")
  .struct("[~ x]*4").s("sawtooth")
  .lpf(400).decay(.2).sustain(.3)
chords: chord("<Cm9 Cm9 Fm9 G7b9>")
  .voicing().struct("~ [~ x] ~ x")
  .s("sawtooth").room(.4)
  .lpf(sine.range(800, 3000).slow(16))
```



batería 1 ciclo · piano roll 2 ciclos: bass · chords

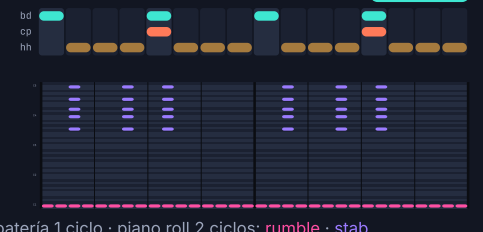
ADN: bombo en negras (four-on-the-floor) · acordes m7/m9 a contratiempo · bajo en la corchea "y"

En vivo: abrí el filtro de los acordes de a poco; sacá el bombo 1 compás antes del drop

Techno

132 BPM

```
setcpm(132/4)
kick: s("bd*4").bank("RolandTR909")
hats: s("[~ hh hh hh]*4")
  .bank("RolandTR909").gain(.4)
clap: s("cp").beat("4,12", 16)
  .room(.3)
rumble: note("c1*16").s("sawtooth")
  .decay(.12).sustain(0)
  .lpf(sine.range(150, 600).slow(8))
stab: chord("Cm").voicing()
  .s("square").beat("2,6,9", 16)
  .decay(.1).sustain(0)
  .delay(.5).room(.5)
```



batería 1 ciclo · piano roll 2 ciclos: rumble · stab

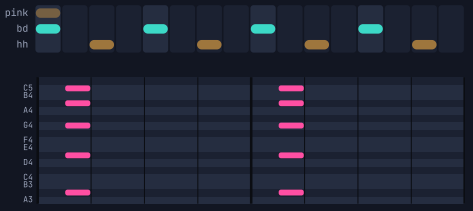
ADN: bombo fuerte y seco · rumble: bajo pegado al bombo · stabs con delay, poca armonía

En vivo: cambiá el .slow() del LFO del rumble; poné .degradeBy(.3) en los hats para el break

Dub Techno

118 BPM

```
setcpm(118/4)
kick: s("bd*4").gain(.9)
hats: s("[~ hh]*4").gain(.35)
chord: chord("<Cm7 Cm7 Fm7 Cm7>")
  .voicing().s("sawtooth")
  .struct("~ x ~ ~ ~ ~ ~ ~")
  .lpf(perlin.range(400, 1500))
  .decay(.15).sustain(0)
  .delay(.7).delaytime(.33)
  .delayfeedback(.7).room(.8)
hiss: s("pink").gain(.03)
```



batería 1 ciclo · piano roll 2 ciclos: chord

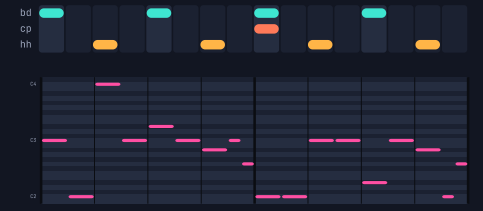
ADN: un acorde m7 que se repite y se disuelve · mucho delay con feedback y reverb · filtro que respira (perlin)

En vivo: subí delayfeedback hacia .8 y bajalo; movés todo el "dub" con un número

Acid

128 BPM

```
setcpm(128/4)
drums: s("bd*4, [~ hh]*4, ~ cp")
  .bank("RolandTR909")
acid: note("c2 c2 c3 c2 eb2 c2 bb1 [c2 g2]")
  .s("sawtooth").ftype('ladder')
  .lpf(sine.range(300, 2000).slow(8))
  .lpq(12).lpenv("<2 4>").lpd(.15)
  .decay(.2).sustain(.2)
  .sometimes(x => x.add(note(12)))
```



batería 1 ciclo · piano roll 2 ciclos: acid

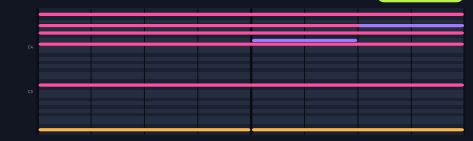
ADN: línea 303: saw + filtro ladder resonante · la envolvente de filtro hace el "squelch" · octavas y acentos aleatorios

En vivo: mové lpq entre 5 y 20 y el LFO del cutoff: eso ES el acid

Ambient

70 BPM

```
setcpm(70/4)
pad: chord("<D^9 Bm11 G^7#11 A6>/2")
  .voicing().s("supersaw")
  .attack(2).release(4).lpf(1200)
  .room(.9).roomsize(8).gain(.5)
bells: n("0 4 7 <9 11>")
  .scale("D:lydian").s("triangle")
  .slow(2).degradeBy(.3)
  .delay(.6).room(.8)
  .pan(perlin.range(.2, .8))
drone: note("d2").s("sine").gain(.4)
```



batería 1 ciclo · piano roll 2 ciclos: pad · bells · drone

ADN: sin bombo: el tiempo flota · acordes largos (attack/release lentos) · notas sueltas con azar y mucho espacio

En vivo: cambiá la escala de las campanas (lydian → dorian) y escuchá cómo cambia el color

PROCESO Cómo construir un tema desde cero

1. **Tempo y tonalidad:** `setcpm(bpm/4)` y elegí una escala (pág. 7).
2. **Bombo:** el esqueleto. Escuchalo solo 8 compases.
3. **Bajo:** que *dialogue* con el bombo (en los huecos) y en la tónica.
4. **Hats/percusión:** el movimiento; acá entra el swing.
5. **Armonía:** 1–4 acordes. Menos es más.
6. **Lead/textura:** una idea melódica corta + azar controlado.
7. **Espacio:** room y delay en sends; `.hpf()` para limpiar.
8. **Forma:** muteá y desmuteá capas (`_nombre:`) cada 8 o 16 compases.

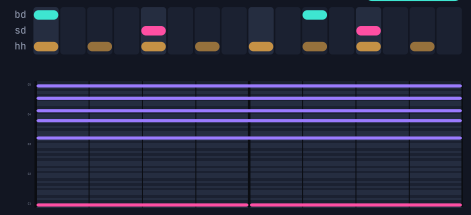
13 Estilos II · DnB, trap, dembow, garage, lo-fi

Más plantillas evaluadas. La tabla del final resume el ADN de cada género: tempo, groove, escala y paleta de sonidos.

Drum & Bass

174 BPM

```
setcpm(174/4)
kick: s("bd").beat("0,10", 16)
snare: s("sd").beat("4,12", 16)
hats: s("hh*8").gain("[.6 .3]*4")
reese: note("<c1 c1 eb1 g0>")
  .s("supersaw").detune(.6)
  .lpf(sine.range(200, 700).slow(4))
  .lpq(6).gain(.7)
pad: chord("<Cm9 Ab^7>/2").voicing()
  .s("triangle").attack(.5)
  .release(2).room(.7).gain(.4)
```



batería 1 ciclo · piano roll 2 ciclos: reese · pad

ADN: breaks rápidos: two-step a 170–175 · bajo reese (saws desafinados) · pads oscuros a medio tiempo

En vivo: poné .every(4, x ⇒ x.ply(2)) en la caja para los fills

Trap / Hip-hop

140 BPM

```
setcpm(140/4)
kick: s("bd").beat("0,7,10", 16)
snare: s("sd").beat("8", 16)
hats: s("hh*8")
  .ply("<1 [1 1 1 2] 1 [1 3 1 1]>")
bass808: note("<c1 c1 eb1 g0>")
  .beat("0,7,10", 16).s("sine")
  .penv(12).pdecay(.04)
  .decay(.9).sustain(0)
mel: n("0 ~ 2 3 ~ 4 3 2")
  .scale("C:harmonic:minor")
  .s("triangle").room(.5).gain(.5)
```



batería 1 ciclo · piano roll 2 ciclos: bass808 · mel

ADN: half-time: caja en el 3 · hats con redobles (ply) · 808 con golpe de pitch y cola larga

En vivo: cambiá los .ply() de los hats (son el alma del trap) y sumá .bank("RolandTR808") a la batería

Dembow / Reggaetón

95 BPM

```
setcpm(95/4)
drums: s("bd*4, [~ ~ ~ sd ~ ~ sd ~]*2")
  .bank("RolandTR808")
hats: s("hh*8").bank("RolandTR808")
  .gain(.5)
bass: note("<a1 f1 c2 g1>")
  .struct("x ~ ~ x ~ ~ x ~")
  .s("sawtooth").lpf(500)
chords: chord("<Am F C G>").voicing()
  .struct("~ x ~ x").s("square")
  .decay(.15).sustain(0)
  .lpf(2000).gain(.5)
```



batería 1 ciclo · piano roll 2 ciclos: bass · chords

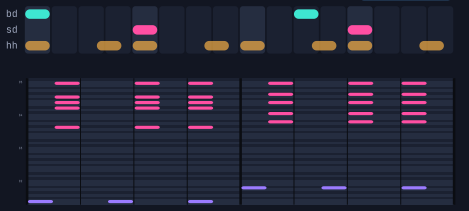
ADN: tresillo 3-3-2 en la caja · progresión menor (i–VI–III–VII) · bajo que sigue el tresillo

En vivo: sacá la caja 1 compás con mask("<1 1 1 0>") antes del coro

UK Garage

132 BPM

```
setcpm(132/4)
kick: s("bd").beat("0,10", 16)
  .bank("RolandTR909")
snare: s("~ sd ~ sd").bank("RolandTR909")
hats: s("hh*8").swingBy(1/3, 4)
  .gain(.5)
organ: chord("<Fm9 Bbm9 Ebm9 Ab^7>")
  .voicing().s("square")
  .struct("~ x ~ ~ x ~ x ~")
  .decay(.12).sustain(0).lpf(2500)
bass: note("<f1 bb1 eb1 ab1>")
  .struct("x ~ ~ x ~ ~ x ~").s("sine")
```



batería 1 ciclo · piano roll 2 ciclos: organ · bass

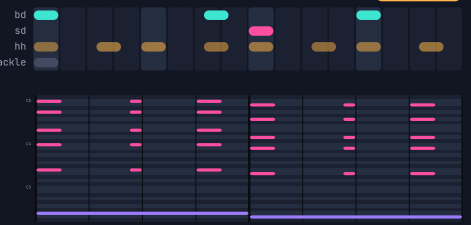
ADN: bombo que saltea, caja en 2 y 4 · swing fuerte en los hats · acordes m9 cortados y sincopados

En vivo: variá el swing: .swingBy(1/6, 4) sutil, 1/3 bien garage

Lo-fi Hip-hop

80 BPM

```
setcpm(80/4)
drums: s("bd ~ ~ bd ~ ~ bd ~, ~ sd")
  .bank("RolandTR808").swingBy(1/6, 4)
hats: s("hh*8").swingBy(1/6, 4)
  .gain(rand.range(.2, .5))
keys: chord("<F^7 Em7 Dm7 C^7>")
  .voicing().s("piano")
  .struct("x ~ ~ [~ x] ~ ~ x ~")
  .lpf(2000).room(.4)
bass: chord("<F^7 Em7 Dm7 C^7>")
  .rootNotes(2).s("sine")
vinyl: s("crackle").gain(.1)
```



batería 1 ciclo · piano roll 2 ciclos: keys · bass

ADN: acordes maj7/m7 (ii–V–I) en piano · batería con swing, todo un poco "sucio" · crackle de vinilo y filtro pasa-bajos

En vivo: bajá el lpf de los keys a 1200 para el puente; subí el crackle

TABLA ADN de los géneros

GÉNERO	BPM	GROOVE	ESCALA	PALETA
House	120–128	4/4 negras	menor · dórico	saws filtrados, órganos
Techno	128–140	4/4 seco	menor · frigio	rumble, stabs, ruido
Dub techno	115–125	4/4 suave	menor (m7)	acorde + delay + reverb
Acid	125–135	4/4	menor	saw + ladder + resonancia
Ambient	60–90	libre	lidio · pentatónica	pads, campanas, drones
DnB	160–180	two-step / breaks	menor	reese, sub, breaks
Trap	130–150 (½)	half-time	menor · m. armónica	808, hats rolls
Reggaetón	88–100	dembow 3-3-2	menor · andaluza	808, plucks, square
UK Garage	130–135	2-step + swing	menor (m9)	órgano, chops, sub
Lo-fi	70–90	boom-bap + swing	maj7/m7 (ii–V–I)	piano, crackle, lpf

14 Disección: el lead del video

El código de la captura (líneas 69–81), ejecutado y desarmado. Cada fila dice qué hace el método y por qué está ahí.

FUENTE El código (versión ejecutable)

```
lead: s("saw").chord("G#m9").voicing()
  .lpf(rand.seed(3).range(200, 1000)).lpe(8)
  .lpa(wchoose(
    [rand.seed("2")
      .range(0.05, 0.1).rib(22, 1), 2],
    ["0", 4],
  ))
  .lpd(rand.range(0.1, 0.7).rib(29, 1))
  .set.out(`<
  ~ ~ 1 _ ~ ~ 1 ~ ~ 1 _ _
>*8`.as("vel"))
  .room(0.4).pan(rand.range(0.4, 0.6))
  .compressor(-10).postgain(0.2).rarely(ply(2))
```

Copiado de la captura, con `.pg` → `.postgain` y sin el `.delay`, que en pantalla aparece cortado.

Lo que ejecuta Strudel (salida real del transpilador, abreviada):

```
s(m('saw', 8))
  .chord(m('G#m9', 21))
  .voicing()
  .lpf(rand.seed(3)
  .range(200, 1000))
  .lpe(8)
  .lpa(wchoose([
    rand.seed(m('2', 115))
  .range(0.05, 0.1)
  .rib(22, 1),
    2
], [
    m('0', 165),
    4
  …
  .p('lead')
```

LÍNEA A LÍNEA Método por método

69	`lead: s("saw")`	Pista **lead**. Oscilador diente de sierra (alias de `sawtooth`).
69	`.chord("G#m9").voicing()`	Acorde de Sol# menor 9; el voicing elige las notas (abajo, en el teclado).
70	`.lpf(rand.seed(3).range(200,1000))`	Cutoff aleatorio por nota, entre 200 y 1000 Hz. Con `seed(3)` es reproducible.
70	`.lpe(8)`	Envolvente de filtro de **8 octavas**: el cutoff se multiplica por 2^8 = 256, así que llega al tope (20 kHz) y en cada nota el filtro se abre del todo.
71-75	`.lpa(wchoose([… rib(22,1), 2], ["0", 4]))`	Attack del filtro: con peso 2 un valor aleatorio entre 0.05 y 0.1 "congelado" con `rib`; con peso 4, ataque 0 (seco). 2 de cada 6 notas "se abren" más suave.
76	`.lpd(rand.range(0.1,0.7) .rib(29,1))`	Decay del filtro aleatorio, pero en loop de 1 ciclo desde el ciclo 29: el mismo "dibujo" de squelch se repite.
77-79	`.set.out(`<~ ~ 1 _ … >*8`.as("vel"))`	**El ritmo.** `.out` = la estructura la manda el patrón de adentro. 12 pasos a 8 por ciclo: la frase dura 1,5 ciclos (polirritmo 12:8).
80	`.room(.4).pan(rand.range (.4,.6)).delay(…)`	Reverb, paneo levemente aleatorio por nota y delay.
81	`.compressor(-10).pg(0.2)`	`pg` **no existe** en @strudel/core 1.2.6. Es casi seguro un alias propio de `postgain`; acá lo reemplazamos por `.postgain(0.2)`.
81	`.rarely(ply(2))`	25% de las veces, cada nota se duplica (redoble).

Ejercicios con este lead. Cambiá una cosa por vez y escuchá:

```
.chord("<G#m9 Em9>")          // 1. dos acordes
.lpe(2)                       // 2. menos "squelch"
.set.out("<1 ~ 1 1>*4".as("vel")) // 3. otro ritmo
.rib(23, 1)                   // 4. otro azar (era 22)
.s("supersaw").detune(.3)     // 5. más ancho
```

POLIRRITMO El ritmo: 12 pasos sobre una grilla de 8



3 ciclos del patrón `<~ ~ 1 _ ~ ~ 1 ~ ~ 1 _ _>*8`. Los grupos son **4+3+5** pasos: los acentos caen en lugares distintos de cada compás y la frase recién se repite cada 3 compases (12 y 8 coinciden cada 24 pasos).

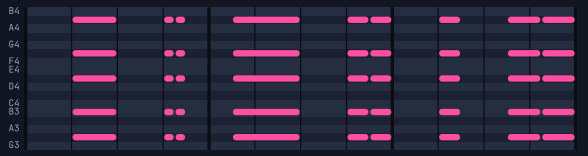
```
m('<~ ~ 1 _ ~ ~ 1>*8', 0).as(m('vel', 23))
```

↑ los backticks también pasan por mini-notación (salida real del transpilador).

RESULTADO La armonía y el resultado



`G#m9` → **Ab3 · B3 · Eb4 · Gb4 · Bb4**: son G# B D# F# A# escritas con bemoles (enarmónicos).



Piano roll de 3 ciclos del lead completo: el acorde entero suena con el ritmo del patrón de velocity.

Lo que podés robar: (1) timbre y ritmo por separado, unidos con `set.out`; (2) azar en el filtro pero congelado con `rib`; (3) `wchoose` para que la mayoría de las notas sean "normales" y algunas especiales.

15 Hydra · visuales en vivo

Hydra (de Olivia Jack) es a los visuales lo que Strudel es al sonido: funciones encadenadas que generan video en la GPU. Strudel ya la trae integrada.

GRAMÁTICA Misma gramática, otro medio

IDEA	STRUDEL (AUDIO)	HYDRA (VIDEO)
fuente	`s("sawtooth")`	`osc(10, .1, 1)`
transformar	`.lpf(800).fast(2)`	`.kaleid(4).rotate(.2)`
modular	`.lpf(sine.range(…))`	`.modulate(noise(3), .2)`
mezclar	`stack(a, b)`	`.add(src) .blend(src)`
salida	(implícita)	`.out(o0)`
tiempo	`ciclos (cps)`	`time (segundos)`
valor vivo	`"<1 2 4>"`	`[1,2,4].fast(1) · () => …`

Se dice **"Hydra"** (/ˈhaɪdrə/ en inglés, "Hidra" en castellano), como la criatura de muchas cabezas.

H() Activarla dentro de Strudel

hydra.mjs:7

```
await initHydra()          // crea el canvas de fondo

osc(30, 0.1, 1.2)          // fuente: ondas de color
  .kaleid(H("<3 4 6 8>"))  // ← un patrón de Strudel
  .rotate(() => time * .1)
  .modulate(noise(3), .2)
  .out()

$: s("bd*4, [~ hh]*4")
$: n("<0 2 4 7>*4").scale("C:minor").s("sawtooth")
```

`H(pat)` devuelve una función que Hydra llama en cada frame; esa función **consulta el patrón en el tiempo actual del scheduler**. Por eso el caleidoscopio cambia de 3 a 8 lados **sincronizado** con la música. `hydra.mjs:51`

OPCIONES Opciones de initHydra

`feedStrudel: 1`	manda el canvas de Strudel (pianoroll, spiral, scope…) a `s0` para procesarlo con Hydra
`detectAudio: true`	opción de Hydra: análisis FFT en `a.fft[0..n]`. Escucha la entrada de audio (micrófono); en Strudel viene apagada
`pixelRatio`	resolución (1 = normal, .5 = más rápido)
`src`	URL de hydra-synth (por defecto unpkg)

```
await initHydra({ feedStrudel: 1 })
src(s0).kaleid(4).modulate(osc(4), .1).out()
$: note("c a f e").s("sawtooth").pianoroll()
```

`clearHydra()` lo apaga. `hydra.mjs:7-51`

API Vocabulario de Hydra

FUENTES `osc(freq, sync, offset)` · `noise(scale, offset)` · `voronoi()` · `shape(lados, radio, suavizado)` · `gradient()` · `solid(r,g,b)` · `src(o0|s0)`

GEOMETRÍA `rotate` · `scale` · `kaleid(n)` · `repeat(x,y)` · `pixelate(x,y)` · `scrollX/Y`

COLOR `color(r,g,b)` · `colorama` · `hue` · `saturate` · `invert` · `brightness` · `contrast` · `posterize` · `thresh` · `luma`

MEZCLA `add` · `sub` · `mult` · `diff` · `blend` · `layer` · `mask`

MODULAR `modulate` · `modulateScale` · `modulateRotate` · `modulateKaleid` · `modulatePixelate` · `modulateHue`

SALIDA `.out(o0…o3)` · `render()` muestra las 4 salidas

RECETAS Recetas por estilo

```
// techno: rejilla que late con el bombo
shape(4, .25, .01).repeat(4, 4)
  .scale(H("<1 1.2>*4")).rotate(.3)
  .color(.2, 1, .8).out()

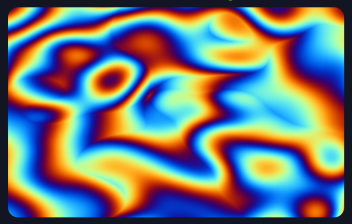
// ambient: nubes lentas
noise(2, .05).color(.4, .6, .9)
  .modulate(osc(1), .3).blend(o0, .9).out()

// dub: feedback (la salida vuelve a entrar)
src(o0).scale(1.01).rotate(.01)
  .blend(osc(20, .1).kaleid(5), .1).out()
```

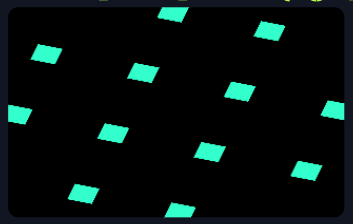
TIPS Puentes mentales

- **Valores que cambian:** en Hydra `[1, 2, 4]` recorre la lista (como `"<1 2 4>"`) y `() => Math.sin(time)` es un LFO (como `sine`).
- **Sincronía real:** usá `H()` con los *mismos* strings de mini-notación que tu música. Así todo lo audiovisual comparte un único reloj.
- **Feedback:** `src(o0)` dentro de `.out(o0)` es el equivalente visual del `delayfeedback`: ecos que se acumulan.
- **Documentación:** hydra.ojack.xyz (editor y docs) y strudel.cc/learn/hydra.

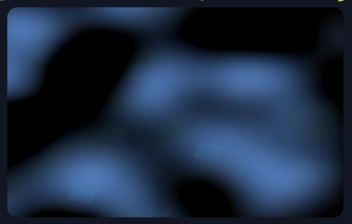
GALERÍA Galería: capturas reales de hydra-synth 1.4 (H() fijado en un valor para la foto)



`osc(30,.1,1.2).kaleid(H(…))`
`.rotate(…).modulate(noise(3),.2)`



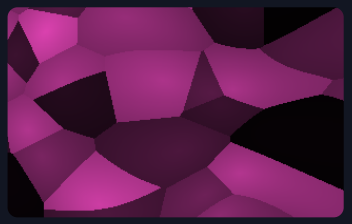
`shape(4,.25,.01).repeat(4,4)`
`.rotate(.3).color(.2,1,.8)`



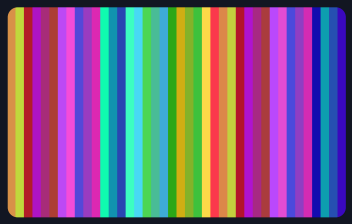
`noise(2,.05).color(.4,.6,.9)`
`.modulate(osc(1),.3).blend(o0,.9)`



`src(o0).scale(1.01).rotate(.01)`
`.blend(osc(20,.1).kaleid(5),.1)`



`voronoi(5,.3,.3).color(1,.3,.8)`
`.modulateScale(osc(8),.5)`



`osc(10,.1,2).colorama(.3)`
`.pixelate(40,25)`

16 Referencia de fuentes + ruta de aprendizaje

Dónde vive cada función en el código (paquetes npm @strudel/* 1.2.6; repo: codeberg.org/uzu/strudel, carpeta packages/). Abrí el archivo y leé la función: tiene la doc y ejemplos encima.

Estructura

stack	pattern.mjs:1321
cat / slowcat	:1454 · :1399
seq / fastcat	:1535 · :1505
arrange	:1469
stepcat / timecat	:3244
polymeter	:2847
silence · pure · reify	:1247 · :1260 · :1285

Tiempo

fast · slow	pattern.mjs:1924 · :1958
early · late	:2061 · :2081
ply	:1905
every / firstOf	:2022
iter · palindrome	:2499 · :2340
chunk	:2571
swingBy · swing	:2184 · :2193
linger · segment	:2151 · :2173
off · jux · rev	:2236 · :2379 · :2259
ribbon / rib	:2678
beat	:3521
euclid · euclidRot	euclid.mjs:138 · :150

Azar y señales

signal · saw · sine	signal.mjs:18 · :35 · :80
rand · irand · perlin	:449 · :488 · :661
choose · wchoose	:532 · :608
seed · shuffle	:436 · :392
degradeBy	:699
sometimes / rarely…	:800 · :867
range · rangex	pattern.mjs:1771 · :1786

Armonía

note · n	controls.mjs:388 · :366
scale	tonal.mjs:268
transpose · scaleTranspose	tonal.mjs:114 · :167
chord · anchor · mode · dict	controls.mjs:2130-2181
voicing · rootNotes	voicings.mjs:196 · :166
arp	pattern.mjs:966

Sonido

s / sound · bank	controls.mjs:110 · :707
attack…release · adsr	:453-758 · :2572
lpf · hpf · bpf	:1204 · :1682 · :772
lpq · lpenv · lpa · lpd	:1705 · :1218 · :1257 · :1296
ftype · vowel	:1425 · :2447
fm · fmh	:473-498
gain · velocity · postgain	:421 · :411 · :431
room · delay · orbit	:2196 · :1733 · :2047
distort · crush · coarse	:2323 · :858 · :868
duckorbit (duck)	:971
as (batch)	:2773

Sistema

Pattern · queryArc	pattern.mjs:45 · :414
register()	pattern.mjs:1605
registerControl()	controls.mjs:63
Hap · TimeSpan	hap.mjs:9 · timespan.mjs:9
composers (set/out/…)	pattern.mjs:1020
.p() / labels	repl.mjs:171 · transpiler.mjs:118
setcpm	repl.mjs:132
initHydra · H	hydra.mjs:7 · :51
cadena de audio	superdough.mjs:682-963

PLAN Ruta sugerida: 8 semanas, 20 minutos por día

SEM	TEMA	QUÉ ESTUDIAR	DESAFÍO
1	**Ritmo**	mini-notación + 3 grooves de la pág. 6	reescribí un groove sin mirar
2	**Transformar**	fast, ply, every, iter, euclid	un loop de 1 compás que dure 3 min sin aburrir
3	**Escalas**	n().scale(), grados, modos	misma melodía en 4 modos: anotá cómo cambia
4	**Acordes**	chord().voicing(), progresiones, rootNotes	tema de 4 acordes con bajo
5	**Síntesis**	ADSR, filtro, lpenv, FM	recreá 3 sonidos de tu sinte de hardware
6	**Azar**	rand, rib, sometimes, perlin	un lead "vivo" a lo autora del video
7	**Mezcla + forma**	orbit, duck, hpf, arrange, mask	tema completo de 3 minutos
8	**Hydra**	initHydra, H(), feedStrudel	tu tema con visuales sincronizados

GLOSARIO Glosario express

ciclo	unidad de tiempo de Strudel (≈ 1 compás)
cps / cpm	ciclos por segundo / minuto (tempo)
hap	un evento: tiempo + valores
control	parámetro de sonido (s, lpf, room…)
señal	patrón continuo (sine, rand)
voicing	disposición de las notas de un acorde
orbit	bus de efectos compartido
cutoff	frecuencia de corte del filtro
resonancia	realce en el cutoff (Q)
envolvente	curva en el tiempo (ADSR)
LFO	oscilador lento que modula un parámetro